

Step 1: Install Node.js

If you do not have Node.js installed on your computer, you need to download and set it up first.

1. Go to the official [Node.js website](https://nodejs.org/).
2. Download the **LTS (Long Term Support)** version for your operating system (Windows, macOS, or Linux).
3. Run the installer and follow the standard setup steps.
4. Verify the installation by opening your terminal or command prompt and running:

```
node -v
```

(If it returns a version number like v20.x or v22.x, Node.js is installed successfully.)

Architecture Overview

Step 2: Initialize the Node.js Client Project

Let's create a dedicated folder for our web client, initialize it, and install the necessary dependencies.

1. Open your terminal or command prompt.
2. Create a new directory and navigate into it:

```
mkdir temp-converter-web-client
```

```
cd temp-converter-web-client
```

3. Initialize a new Node.js project:

```
npm init -y
```

4. Install express, which is the web framework used to host the client:

Bash

```
npm install express
```

Step 3: Create the Node.js Proxy Server (server.js)

In the temp-converter-web-client directory, create a file named **server.js** using a text editor (like Notepad or VS Code) and paste the following code:

```
const express = require('express');
const path = require('path');

const app = express();
const PORT = 3000;

// Serve static files (like index.html) from the project folder
app.use(express.static(__dirname));

// Accept incoming JSON payloads
app.use(express.json());

// Proxy Endpoint: Handles requests from your HTML page and forwards them
app.post('/api/convert', async (req, res) => {
  const { value, unit } = req.query;

  try {
    // Node.js makes an HTTP call to the Spring Boot application (backend to backend)
    const response = await
fetch(`http://localhost:8080/api/temperatures/convert?value=${value}&unit=${unit}`,
{
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  }
});

    if (!response.ok) {
      return res.status(response.status).json({ error: 'Failed to communicate
with Spring Boot' });
    }

    const data = await response.json();

    // Return the resulting data to the frontend
    res.json(data);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

app.listen(PORT, () => {
  console.log(`Server running and accessible at: http://localhost:${PORT}`);
});
```

Step 4: Create the HTML Client (index.html).In the same folder as server.js, create a file named index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Temperature Converter Client</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f7fc;
      display: flex;
      justify-content: center;
      padding-top: 50px;
    }
    .container {
      background: white;
      padding: 38px;
      border-radius: 8px;
      border: 1px solid #ccc;
      box-shadow: 0 4px 8px rgba(0,0,0,0.1);
      width: 350px;
    }
    h2 {
      text-align: center;
      color: #333;
      margin-bottom: 20px;
    }
    .form-group {
      margin-bottom: 15px;
    }
    label {
      display: block;
      margin-bottom: 5px;
      font-weight: bold;
      color: #333;
    }
    input, select, button {
      width: 100%;
      padding: 10px;
      margin-top: 5px;
      margin-bottom: 5px;
      box-sizing: border-box;
      border: 1px solid #ccc;
      border-radius: 4px;
    }
    button {
      background-color: #8070ff;
      color: white;
      border: none;
      font-weight: bold;
      cursor: pointer;
      margin-top: 10px;
    }
    button:hover {
      background-color: #8060ff;
    }
    .output-box {
      margin-top: 20px;
      padding: 15px;
      background-color: #f9f9f9;
      border-radius: 4px;
      font-family: monospace;
      min-height: 60px;
      white-space: pre-wrap;
      color: #333;
    }
  </style>
</head>
<body>

  <div class="container">
    <h2>Temperature Converter</h2>

    <div class="form-group">
      <label for="tempValue">Temperature Value</label>
      <input type="number" id="tempValue" placeholder="e.g. 100" required>
    </div>

    <div class="form-group">
      <label for="tempUnit">Select Unit</label>
      <select id="tempUnit">
        <option value="C">Celsius (C)</option>
        <option value="F">Fahrenheit (F)</option>
      </select>
    </div>

    <button onclick="convertTemperature()">Convert</button>

    <div class="output-box" id="outputArea">Result will appear here...</div>
  </div>

  <script>
    // Point the client to the local Node.js server instead of Spring Boot
    const API_URL = '/api/convert';

    async function convertTemperature() {
      const valueInput = document.getElementById('tempValue').value;
      const unitInput = document.getElementById('tempUnit').value;
      const outputArea = document.getElementById('outputArea');

      if (!valueInput) {
        outputArea.textContent = "Error: Please enter a numeric temperature";
        return;
      }

      outputArea.textContent = "Converting...";

      try {
        // Fetch the API endpoint exposed on the Node.js server
        const response = await fetch(`${API_URL}?value=${valueInput}&unit=${unitInput}`, {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json'
          }
        });

        if (!response.ok) {
          throw new Error(`Server Error: ${response.status} - ${response.statusText}`);
        }

        const data = await response.json();
        outputArea.textContent = `Conversion Successful!\n\n` +
          `ID: ${data.id}\n` +
          `Value: ${data.value} ${data.unit}\n` +
          `Result: ${data.result}\n` +
          `Timestamp: ${data.timestamp}`;

      } catch (error) {
        outputArea.textContent = `Error: ${error.message}`;
      }
    }
  </script>
</body>
</html>
```

Step 5: Run the Complete Setup

1. **Start your Spring Boot API:** Ensure your Spring Boot application is running and listening on port 8080.

(You do not need to add any CORS annotations for this proxy approach).

2. **Start the Node.js Client:** Open the terminal in the temp-converter-web-client directory and run:

Bash

```
node server.js
```

3. Open your web browser and go to **http://localhost:3000** to test the application.